

1. Spam Detection – Problem (7 Marks)

The core problem of spam detection is a **binary classification task** in machine learning: given an incoming email (or message), classify it as either "**spam**" (unwanted, unsolicited message) or "**ham**" (legitimate, non-spam message).

Key Challenges

- **High Volume and Speed:** Spammers constantly generate new content, requiring detection systems to be fast, highly accurate, and adaptable to real-time data streams.
- **Adversarial Environment:** Spammers actively try to **evade detection** by using techniques like:
 - **Obfuscation:** Intentionally misspelling words (e.g., "V1agra" instead of "Viagra").
 - **Image-based Spam:** Embedding text within images, making it difficult for traditional text-based filters to read.
 - **Polymorphism:** Varying the email content and structure across messages to avoid being flagged by simple signature matches.
- **High Cost of Error:** The two types of errors have different impacts:
 - **False Positive (Type I Error):** A legitimate email (**ham**) is classified as **spam**. This is the **more serious error**, as it can lead to missed opportunities or important information.
 - **False Negative (Type II Error):** A spam email is classified as **ham**. This is annoying but less damaging than a false positive.
- **Class Imbalance:** In a typical inbox, the number of **ham** emails significantly outweighs the number of **spam** emails. This imbalance can bias a classifier towards the majority class ("ham").

The solution requires a system that can **learn patterns** from the message content (text, headers, etc.) and classify new, unseen messages with high **precision** and **recall**, especially on the "ham" class.

2. Naive Bayes Theorem (7 Marks)

The Naive Bayes classifier is a powerful family of algorithms based on **Bayes' Theorem**. It is particularly well-suited for classification problems involving a large number of features, like text.

The Theorem

Bayes' theorem provides a way to calculate the **posterior probability**, $P(A|B)$, from the **likelihood**, $P(B|A)$, and the **prior probability**, $P(A)$.

$$P(\text{Class}|\text{Features}) = \frac{P(\text{Features}|\text{Class}) \cdot P(\text{Class})}{P(\text{Features})}$$

In the context of spam detection:

- Class is the message category (Spam or Ham).
- Features are the words in the email message $(w_1, w_2, \dots, w_n)$.
- $P(\text{Spam}|\text{Words})$: **Posterior Probability** – The probability that a message is **Spam** given the specific set of **Words** it contains. This is what we want to find.
- $P(\text{Words}|\text{Spam})$: **Likelihood** – The probability of observing this specific set of **Words** if the message is **Spam**.
- $P(\text{Spam})$: **Prior Probability** – The overall proportion of **Spam** messages in the entire dataset.

The "Naive" Assumption

The Naive Bayes algorithm introduces a simplifying "**naive**" **assumption**: that all features (words) are **conditionally independent** of each other given the class.

$$P(\text{Words}|\text{Class}) = P(w_1, w_2, \dots, w_n|\text{Class}) \approx P(w_1|\text{Class}) \cdot P(w_2|\text{Class}) \cdot \dots \cdot P(w_n|\text{Class})$$

While this assumption is almost always false in real-world text (the presence of "buy" is certainly related to the presence of "now"), the classifier often performs surprisingly well, especially in text classification.

3. Intuition (7 Marks)

The intuition behind using Naive Bayes for spam detection is to leverage the **difference in word usage** between spam and legitimate (ham) messages. The model learns which words are *characteristic* of each class.

Core Idea: Probability Scoring

Imagine the model assigns a "score" to a new email for being spam and a score for being ham. It then classifies the email based on the **higher score**.

1. **Training Phase (Learning):** The model scans thousands of labeled emails to build two distinct probability profiles:
 - **Spam Profile:** It calculates the frequency of every word (e.g., "free," "viagra," "lottery," "unsubscribe") that appears in **spam** emails, converting them into likelihoods $P(\text{word}|\text{Spam})$.
 - **Ham Profile:** Similarly, it calculates the frequency of words (e.g., "meeting," "attached," "regards," "project") that appear in **ham** emails, resulting in $P(\text{word}|\text{Ham})$.
2. **Classification Phase (Prediction):** For a new, unseen email containing the words $\{w_1, w_2, w_3, \dots\}$:
 - The model calculates the total probability of the email being **Spam** by multiplying the prior probability of spam, $P(\text{Spam})$, with the likelihoods of all the email's words appearing in a spam message: $P(\text{Spam}) \times P(w_1|\text{Spam}) \times P(w_2|\text{Spam}) \times \dots$
 - It does the same for the **Ham** class.
 - **Decision:** If the Spam score is **greater** than the Ham score, the email is classified as **Spam**.

Example: If an email contains the word "**free**," and $P(\text{free}|\text{Spam})$ is 10x higher than $P(\text{free}|\text{Ham})$, the word "**free**" contributes heavily towards the message being classified as spam. Conversely, the word "**meeting**" might contribute heavily towards it being ham.

4. Spam Detection using Naive Bayes (7 Marks)

Implementing spam detection with Naive Bayes involves several data preprocessing and modeling steps, typically using the **Multinomial Naive Bayes** variant for text classification.

Steps

1. **Data Preparation:** Collect a dataset of emails, each labeled as either "spam" or "ham."
2. **Text Preprocessing:**
 - **Tokenization:** Breaking down the email text into individual words or tokens.
 - **Normalization:** Converting all text to lowercase.
 - **Stop Word Removal:** Removing common, non-informative words like "the," "a," "is," which don't help differentiate between classes.
 - **Stemming/Lemmatization (Optional):** Reducing words to their root form (e.g., "running" to "run").
3. **Feature Engineering (Vectorization):** The classifier can't process raw text, so the text must be converted into numerical features.
 - **Bag-of-Words (BoW) Model:** This is the common approach. The vocabulary of all unique words is extracted. Each email is then represented as a vector (a row in a matrix) where each column corresponds to a word in the vocabulary, and the cell value is the **count** of that word in the email (or a **TF-IDF** score).
4. **Model Training:** The Multinomial Naive Bayes classifier is trained on the numerical feature matrix. The training process essentially calculates all the necessary prior probabilities $P(\text{Class})$ and likelihoods $P(\text{word}|\text{Class})$ from the dataset.
5. **Smoothing (Laplace Smoothing):** To prevent zero probabilities, which can nullify the entire product, **Laplace Smoothing** (adding a small constant, often 1, to every word count) is used. This ensures that even words not seen in the training data can still have a non-zero, albeit very small, probability.
6. **Prediction:** The trained model is used to predict the class of new, unseen emails.

5. Class Imbalance (7 Marks)

Class Imbalance is a common problem in spam detection where one class (the **majority class**) has significantly more instances than the other (the **minority class**). In this case, **Ham** is the majority class, and **Spam** is the minority class.

Impact on Model Performance

A model trained on imbalanced data tends to be biased towards the majority class. It might achieve a high overall **Accuracy** simply by classifying everything as "Ham." For example, if 95% of emails are Ham, a model that classifies **every email** as Ham will still have 95% accuracy, which is misleading and unacceptable for a practical spam filter.

Solutions to Address Imbalance

1. Data-Level Techniques:

- **Oversampling (Minority Class):** Creating synthetic or duplicate copies of the minority class (Spam) samples (e.g., using **SMOTE** - Synthetic Minority Over-sampling Technique).
- **Undersampling (Majority Class):** Randomly removing samples from the majority class (Ham) to balance the ratio. This can lead to a loss of important information.

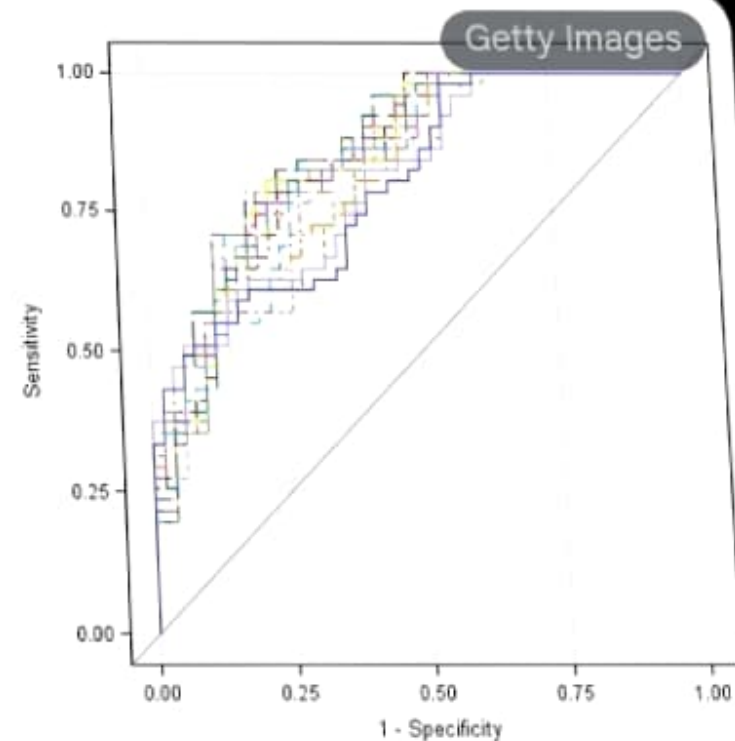
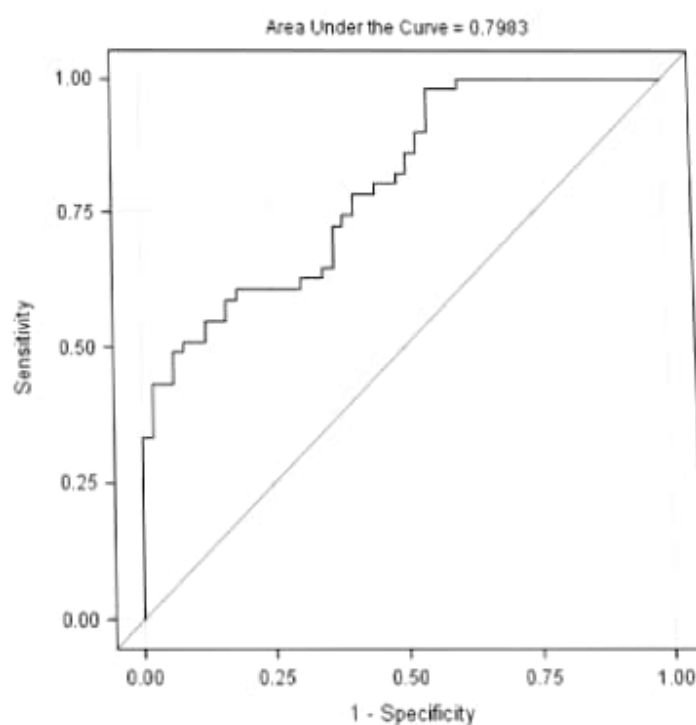
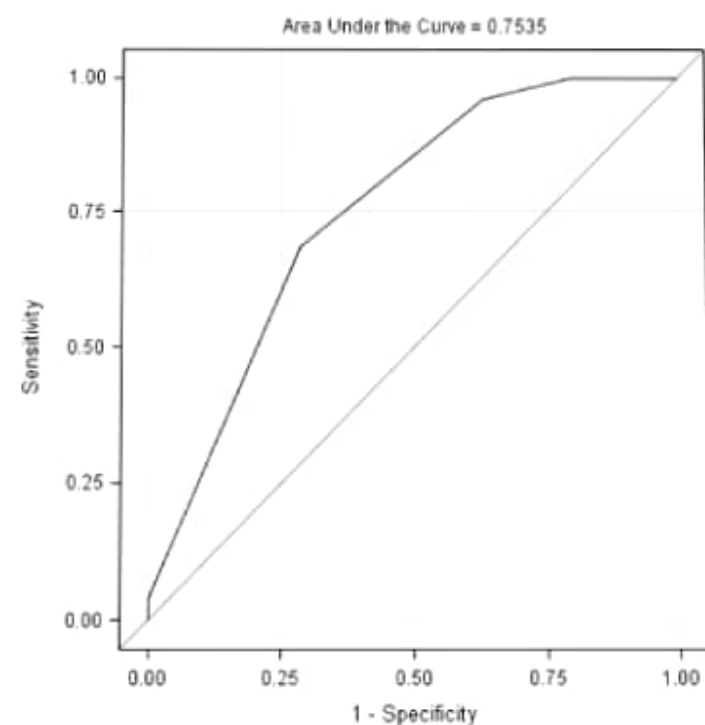
2. Algorithm-Level Techniques:

- **Cost-Sensitive Learning:** Assigning a higher misclassification **cost** to errors made on the minority class (False Negatives for Ham, but most importantly **False Positives for Spam**—i.e., a missed Spam is less costly than a blocked Ham).
- **Use of Appropriate Metrics:** Shifting focus from simple **Accuracy** to metrics that are robust to imbalance, such as **Precision, Recall, F1-Score, and AUC-ROC**.

6. ROC, AUC, and F1 Score (7 Marks)

These are crucial evaluation metrics for binary classification models, especially in the presence of class imbalance.

ROC (Receiver Operating Characteristic) Curve



AUC (Area Under the ROC Curve)

- **Definition:** A single scalar value that summarizes the performance of the classifier across all possible classification thresholds. It is the area beneath the ROC curve.
- **Interpretation:**
 - **AUC = 1:** Perfect classifier.
 - **AUC = 0.5:** Random guessing.
 - **AUC > 0.7:** Generally considered acceptable performance.
- **Use:** AUC is a **threshold-independent** metric, meaning it tells you how well the model can rank predictions, regardless of where you set the decision boundary.

F1 Score

- **Definition:** The **harmonic mean** of **Precision** and **Recall**. It is a metric that balances both concerns.
- $$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$
- **Precision:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$ (Out of all predicted spam, how many were actually spam?)
- **Recall:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$ (Out of all actual spam, how many did we correctly identify?)
- **Use:** The F1 Score is particularly useful for **imbalanced datasets** because it penalizes models that have poor performance in either Precision or Recall, providing a more balanced view of the classifier's performance on the minority class (Spam).